

Lecture 5: **optimization**

what actually happens inside `scipy.optimize`?

September 9, 2026

assignment:

probabilistically forecast F_{new} for a follow up experiment to “Example 1: Less Simple Photon Count” — *i.e.* plot the posterior predictive distribution

posterior predictive:
$$p(x \mid X) = \int d\theta \, p(x \mid \theta) \, p(\theta \mid X)$$

don't forget to normalize the posterior

in-class presentation sign-up sheet: <https://tinyurl.com/ast384c>

15 minute presentation on an astro/stat/ML paper that applies the method covered in the last class — seminal paper or a paper that builds on the method

slides, journal club, presentation slides, whatever format that effectively conveys information

Lec4_bayes_demo.ipynb — we used `scipy.optimize.fmin` to estimate MLE and MAP

```
:  
# maximizing likelihood is equivalent to  
# minimizing negative log likelihood  
def neg_log_likelihood(theta, F, e):  
    return -log_likelihood(theta, F, e)  
  
theta_guess = [900, 20] # initial guess  
theta_est = sp.optimize.fmin(neg_log_likelihood,  
                             theta_guess,  
                             args=(F, e))  
  
print("""  
    Maximum likelihood estimate for {0} data points:  
        mu={theta[0]:.0f}, sigma={theta[1]:.0f}  
    """).format(N, theta=theta_est))
```

optimization

$$\theta^* = \operatorname{argmin}_{\theta} f(\theta)$$

for some objective function $f(\theta)$

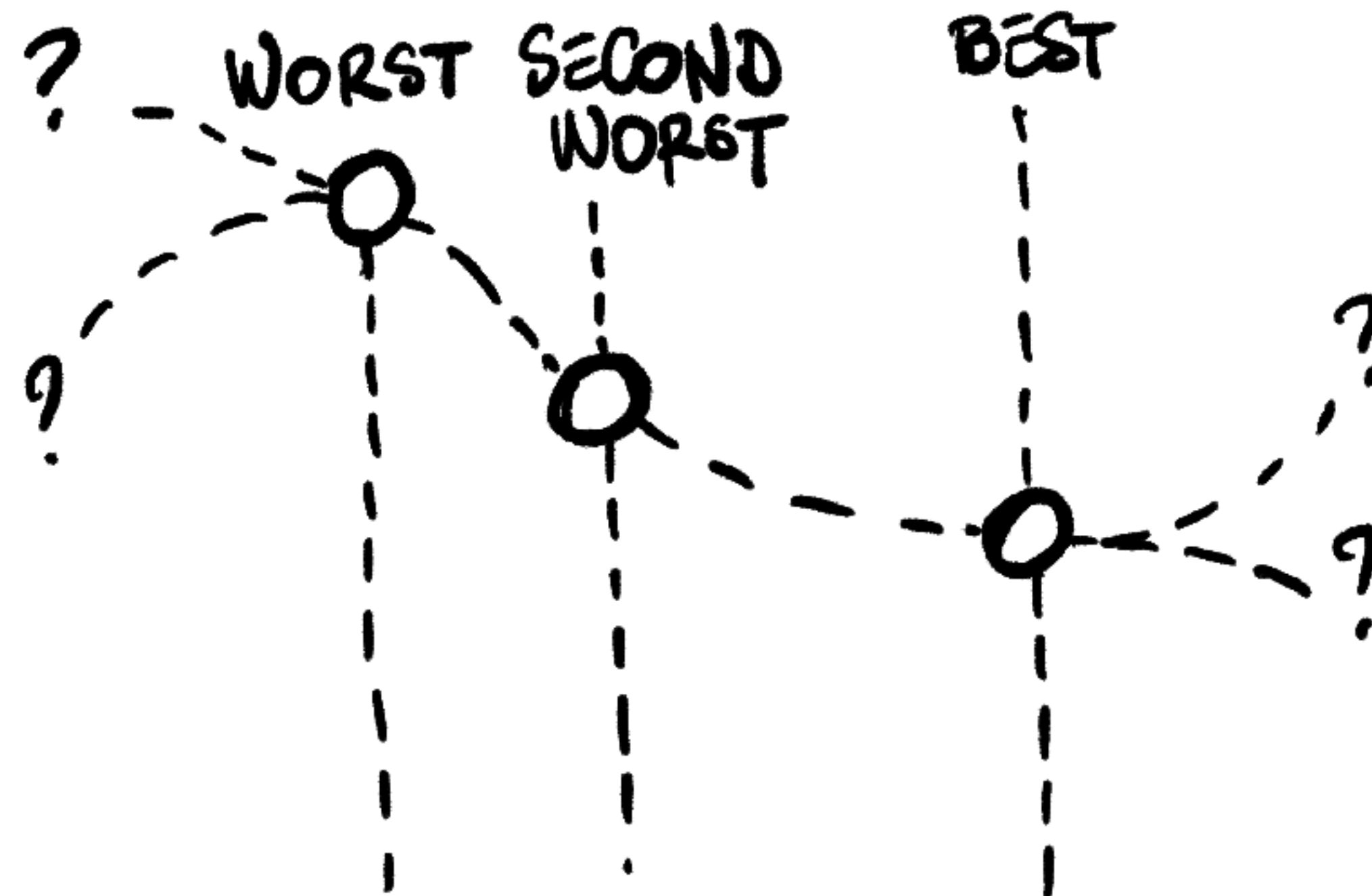
maximum likelihood estimate (MLE): $-\log p(X | \theta)$

maximum a posteriori (MAP): $-\log p(\theta | X)$

different optimization algorithms

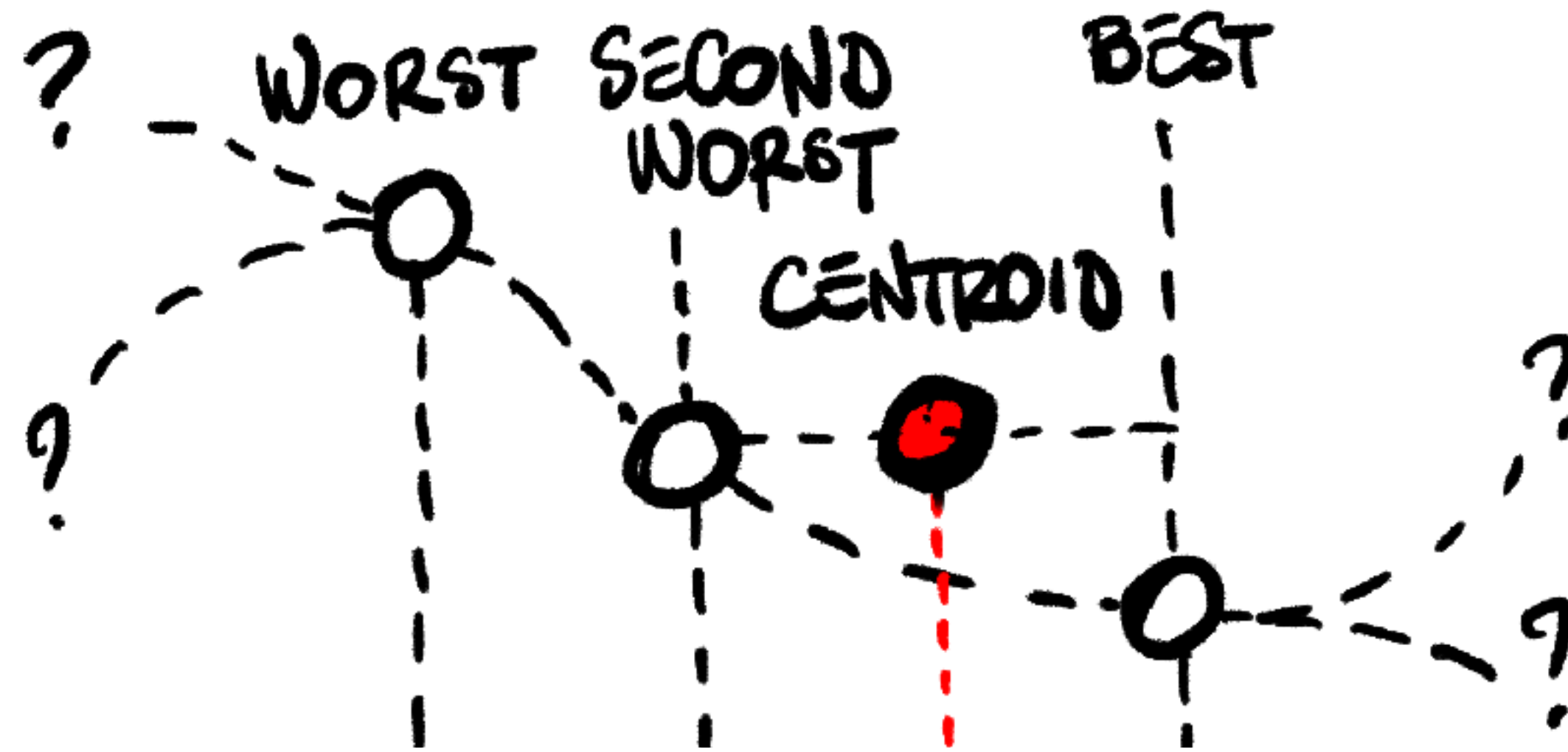
Nelder-Mead — what `scipy.optimize.fmin` actually runs

Nelder-Mead — what `scipy.optimize.fmin` actually runs



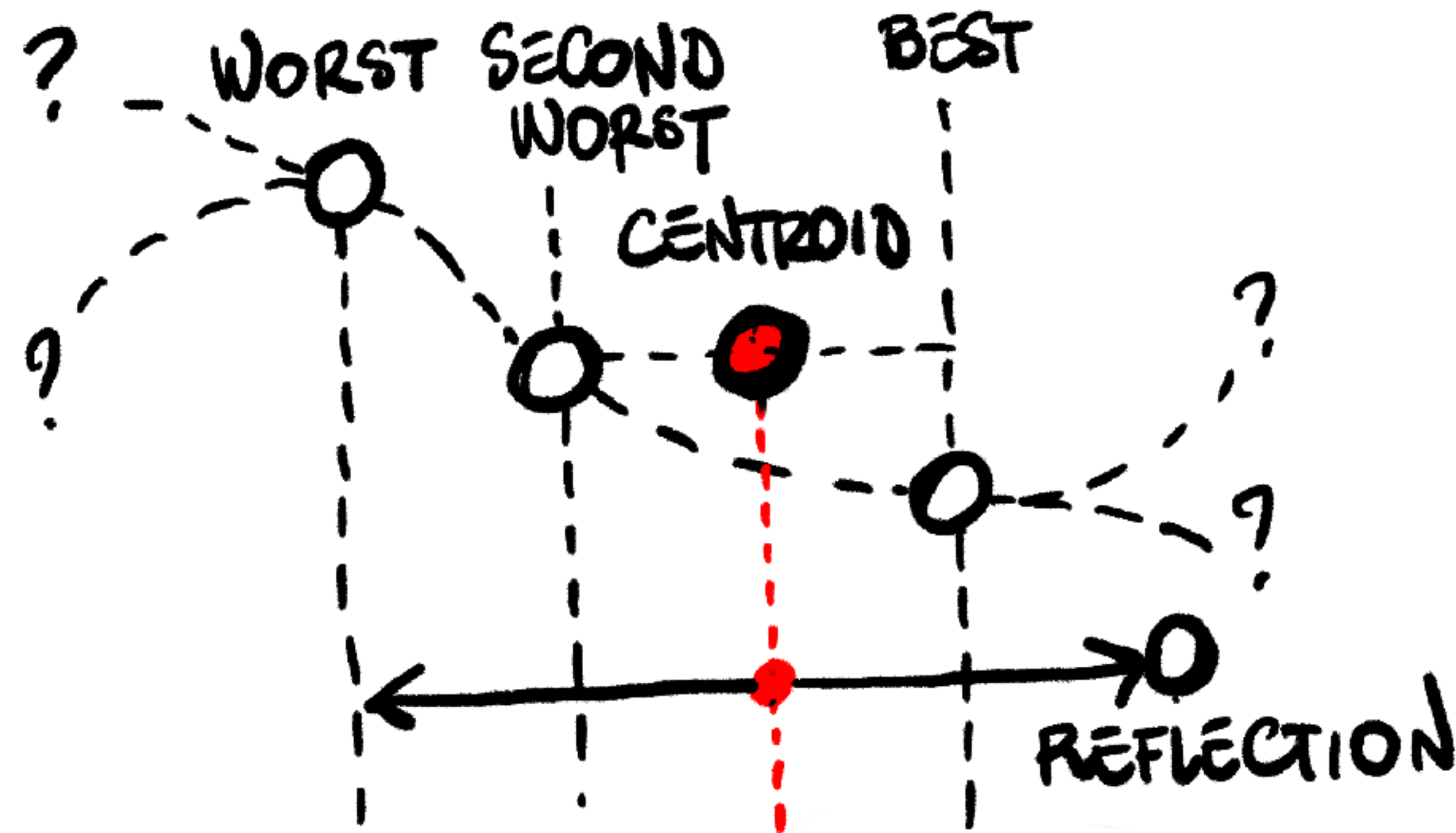
evaluate $f(\theta)$ at candidate values (the **simplex**)

Nelder-Mead — what `scipy.optimize.fmin` actually runs



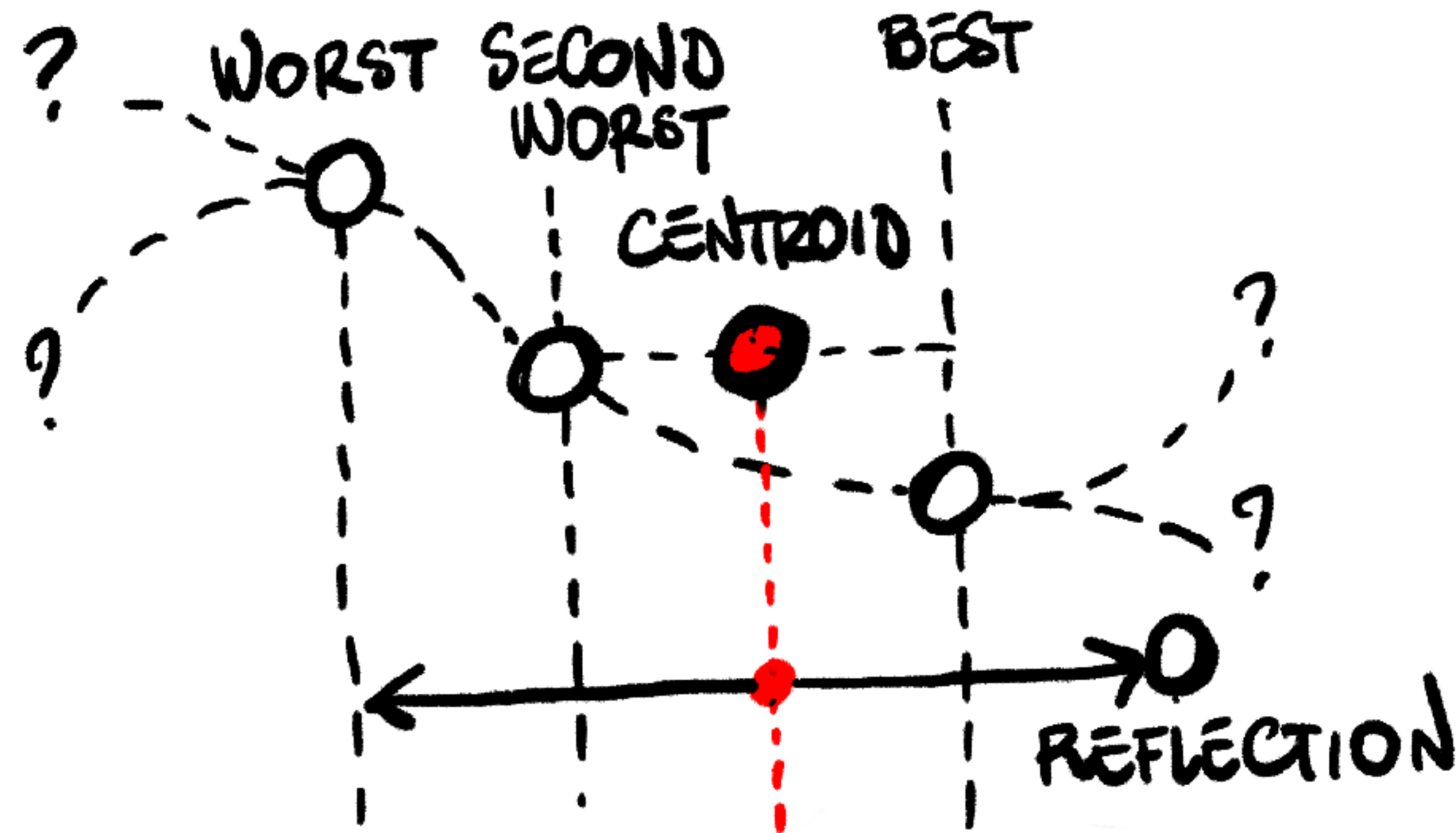
evaluate the **centroid** of all but the worst candidate values

Nelder-Mead — what `scipy.optimize.fmin` actually runs



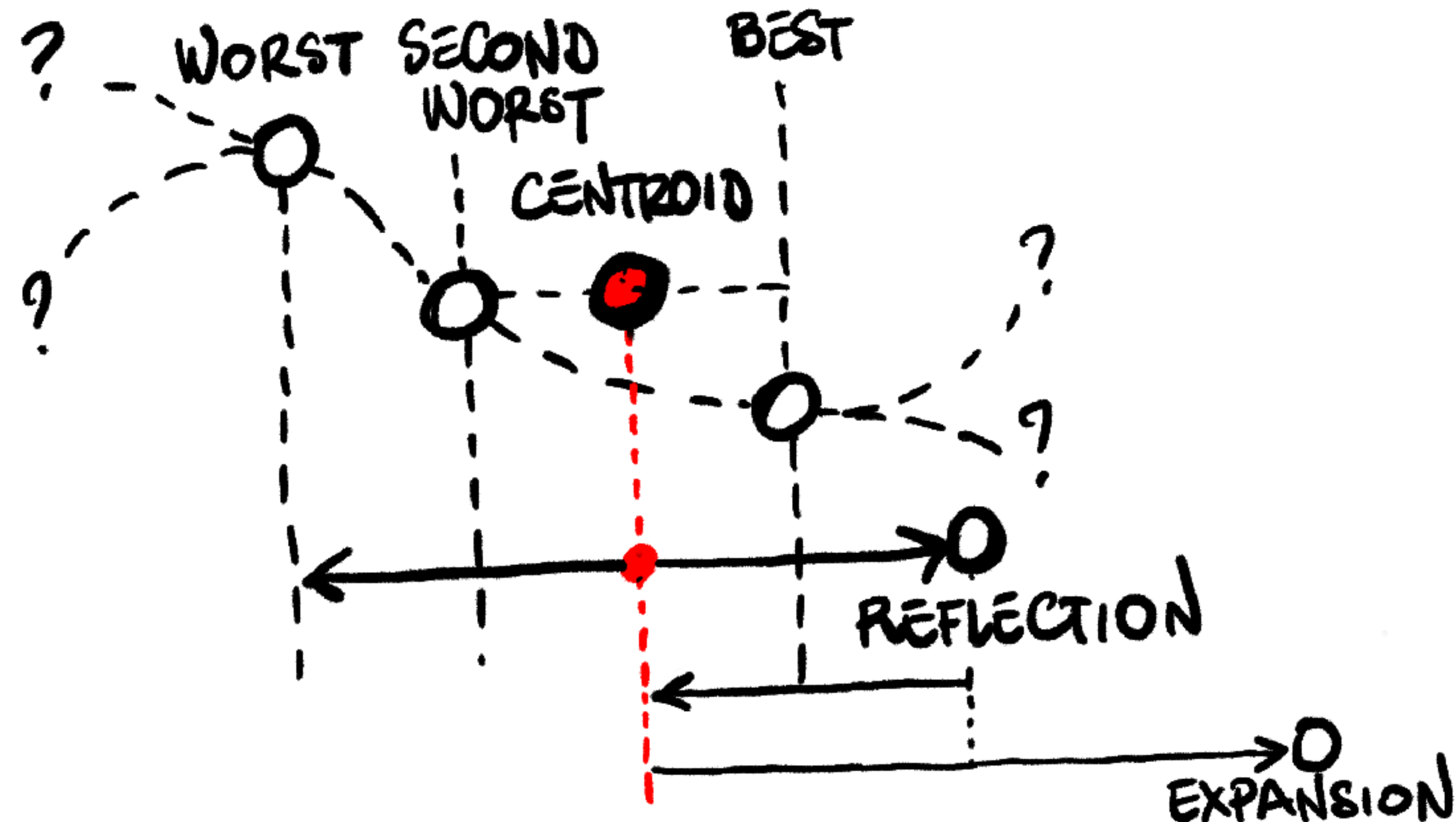
reflection: move the worst candidate to the other side of the centroid

Nelder-Mead — what `scipy.optimize.fmin` actually runs



reflection: move the worst candidate to the other side of the centroid
if reflection is better than the second worst but not better than the best, accept

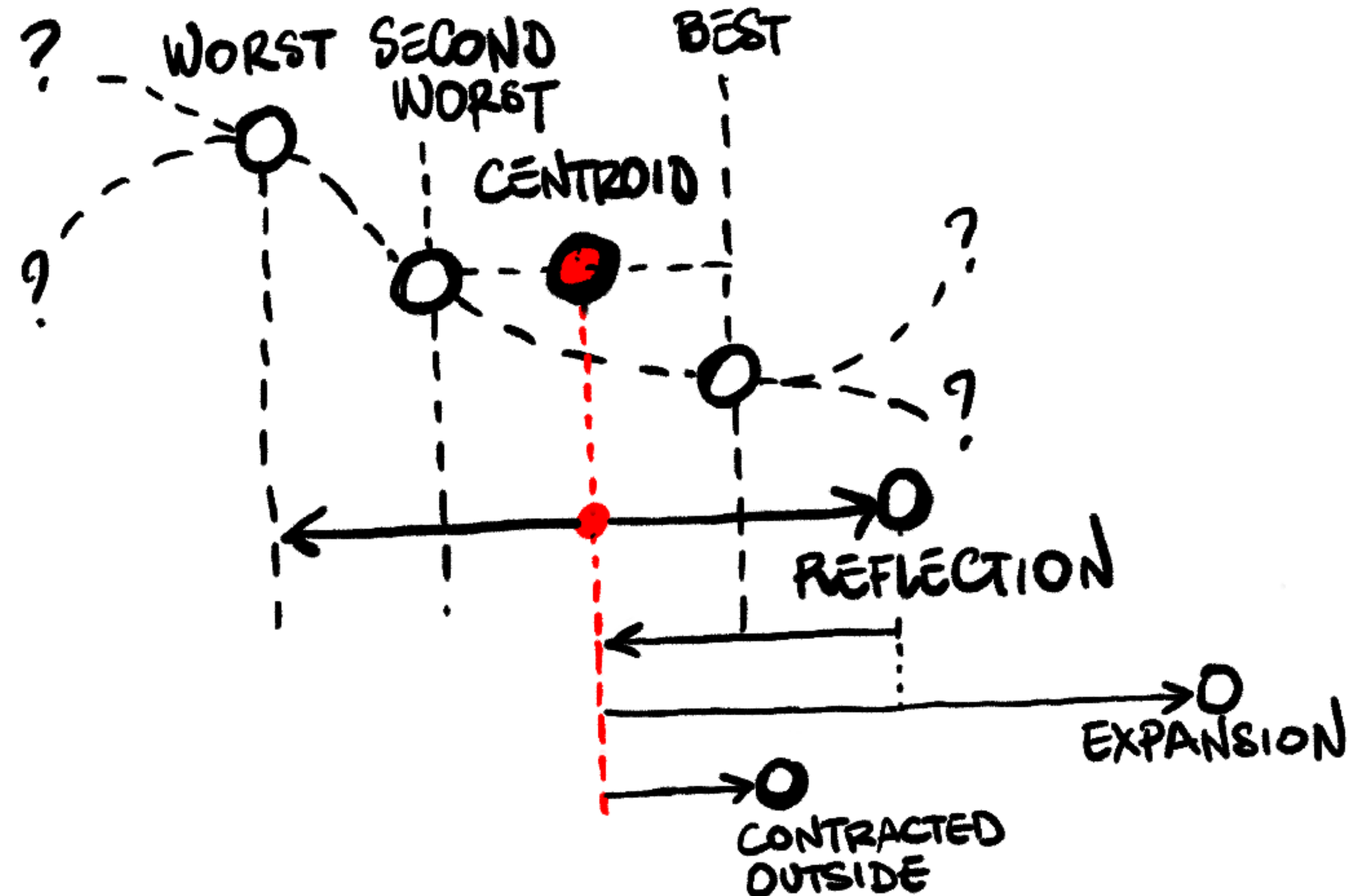
Nelder-Mead — what `scipy.optimize.fmin` actually runs



if reflection is better than the best, **expand** further and keep whichever is best

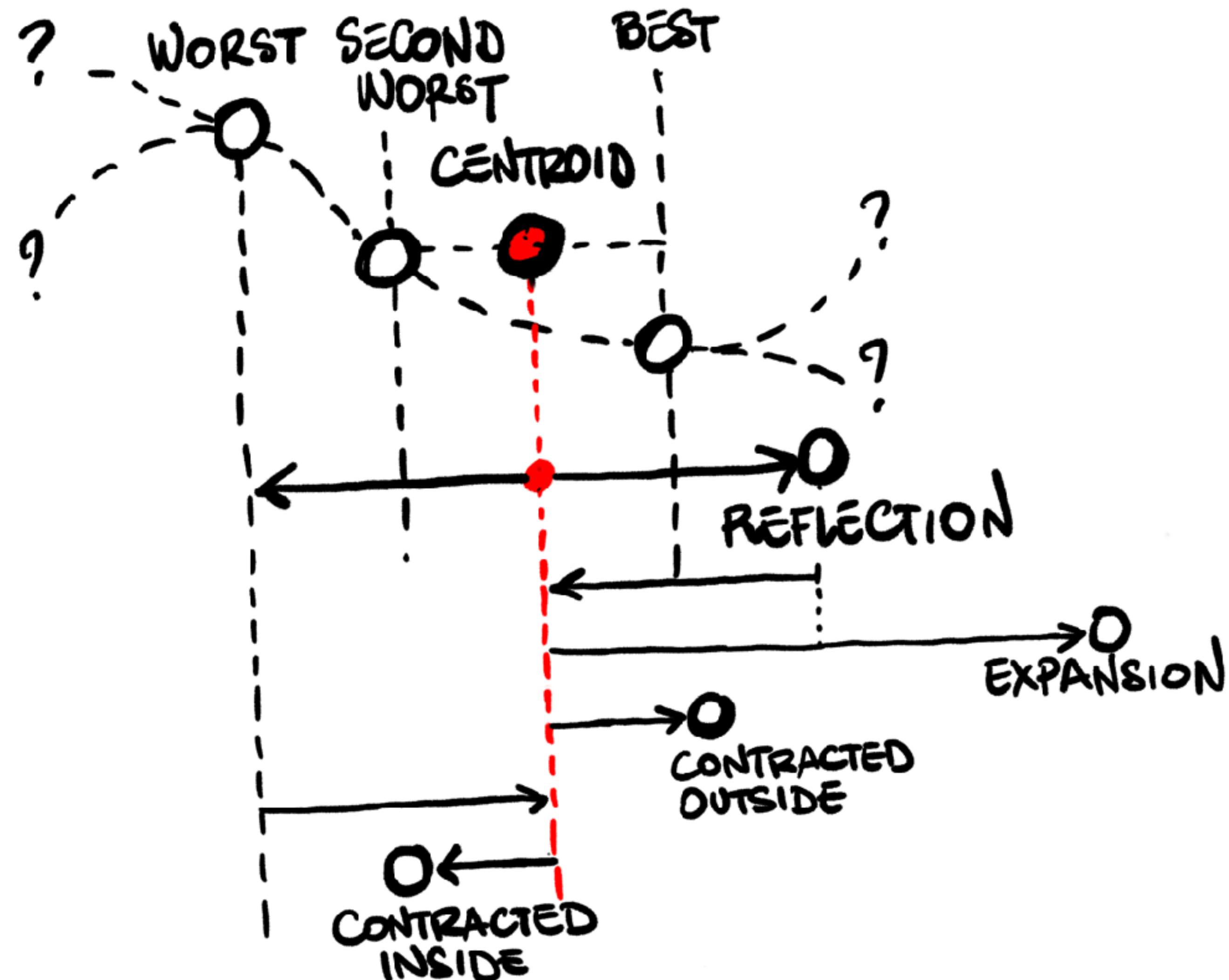
Nelder-Mead — what `scipy.optimize.fmin` actually runs

if reflection is worse than the second best, contracted outside



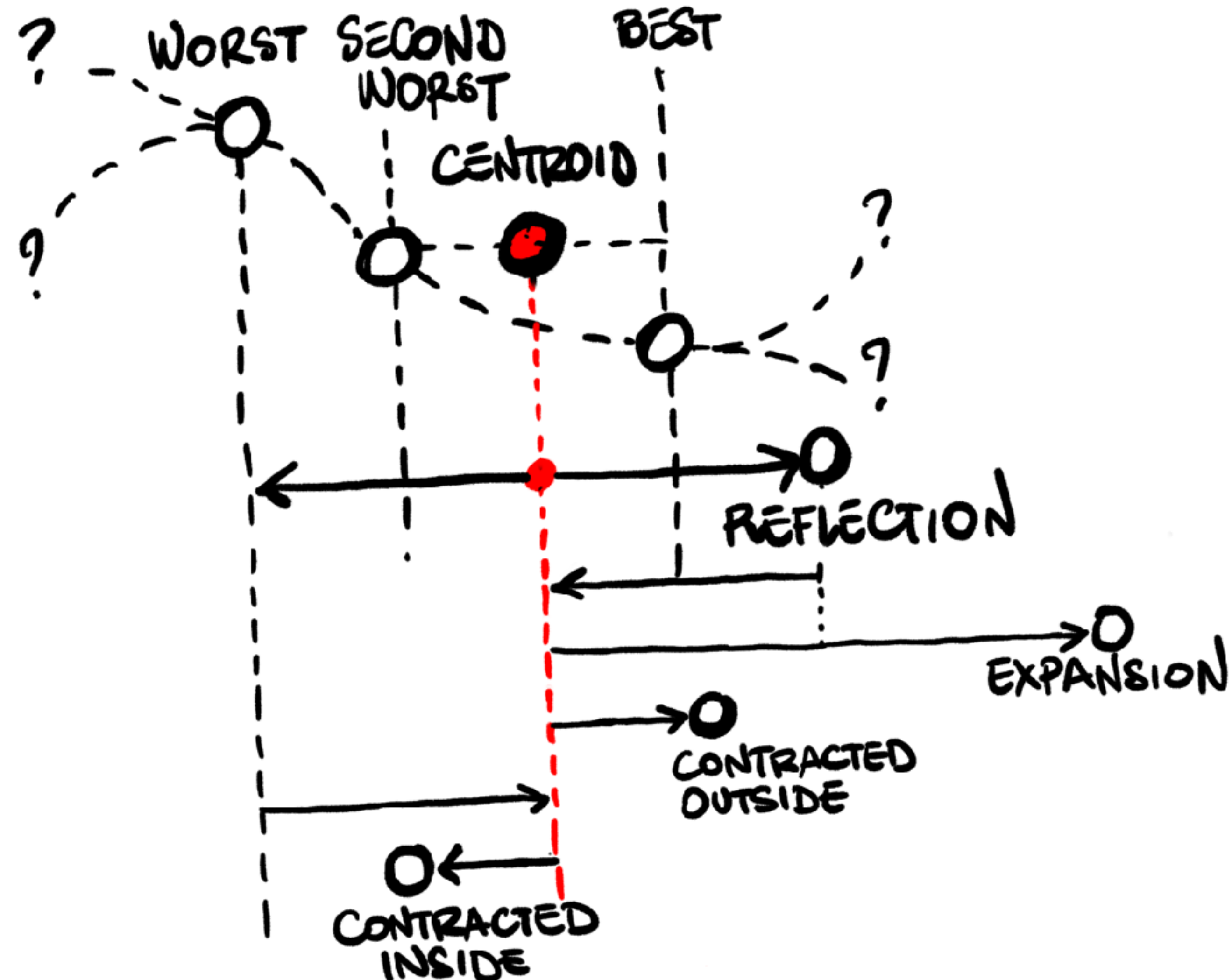
Nelder-Mead — what `scipy.optimize.fmin` actually runs

if reflection is worse than the worst, contracted inside



Nelder-Mead — what `scipy.optimize.fmin` actually runs

if contractions fail **shrink**: move every point towards the best candidate



Nelder-Mead — what `scipy.optimize.fmin` actually runs

```
Initialize: Simplex of  $n+1$  vertices  $\{x_1, \dots, x_{n+1}\}$  near initial guess  $x_0$ 
```

```
Loop until convergence ( $\text{std\_dev}(f(x)) < \text{tol}$ ):
```

1. Order: Sort vertices such that $f(x_1) \leq f(x_2) \leq \dots \leq f(x_{n+1})$
2. Centroid: $x_o = \text{mean}(x_1 \text{ to } x_n)$ [Exclude x_{n+1}]

3. Transform:

```
Compute Reflected:  $x_r = x_o + \alpha(x_o - x_{n+1})$ 
```

```
IF  $f(x_1) \leq f(x_r) < f(x_n)$ : // 3A. Accept Reflection  
 $x_{n+1} = x_r$ 
```

```
ELSE IF  $f(x_r) < f(x_1)$ : // 3B. Try Expansion  
 $x_e = x_o + \gamma(x_r - x_o)$   
 $x_{n+1} = (f(x_e) < f(x_r)) ? x_e : x_r$ 
```

```
ELSE IF  $f(x_r) < f(x_{n+1})$ : // 3C. Outside Contraction  
 $x_c = x_o + \beta(x_r - x_o)$   
IF  $f(x_c) \leq f(x_r)$ :  $x_{n+1} = x_c$  ELSE: Go to Shrink
```

```
ELSE: // 3D. Inside Contraction  
 $x_c = x_o - \beta(x_o - x_{n+1})$   
IF  $f(x_c) < f(x_{n+1})$ :  $x_{n+1} = x_c$  ELSE: Go to Shrink
```

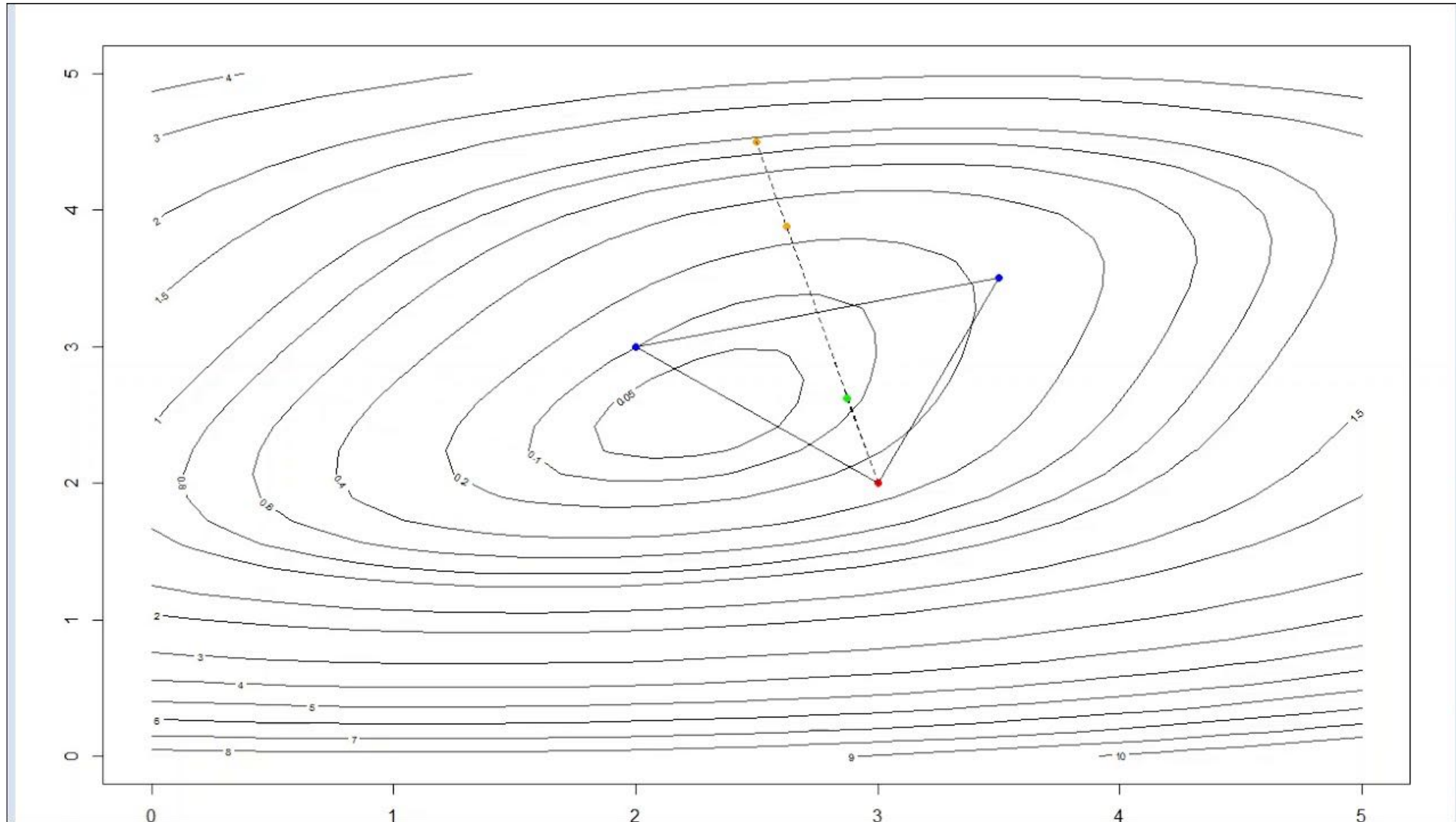
```
Continue to next iteration
```

4. Shrink: (Only triggers if Contraction fails)

```
For  $i = 2$  to  $n+1$ :  
 $x_i = x_1 + \delta(x_i - x_1)$ 
```

```
Return  $x_1$  (best minimum found)
```

Nelder-Mead — what `scipy.optimize.fmin` actually runs



Nelder-Mead — when and when not to use it

needs only functional evaluations — **no derivatives**

~simple but slow and scales poor at high dimensions

no convergence guarantees

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \nabla f(\theta_0)^T (\theta - \theta_0) + \frac{1}{2}(\theta - \theta_0)^T \mathbf{H} f(\theta_0) (\theta - \theta_0) + \mathcal{O}((\theta - \theta_0)^3)$$

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \nabla f(\theta_0)^T (\theta - \theta_0) + \frac{1}{2} (\theta - \theta_0)^T \mathbf{H} f(\theta_0) (\theta - \theta_0) + \mathcal{O}((\theta - \theta_0)^3)$$

gradient

hessian

$$\frac{\partial f}{\partial \theta}$$

$$H_{ij} = \frac{\partial^2 f}{\partial \theta_i \partial \theta_j}$$

gradient descent

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \nabla f(\theta_0)^T (\theta - \theta_0)$$

only keep the first order

gradient

gradient descent

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \nabla f(\theta_0)^T (\theta - \theta_0)$$

only keep the first order

gradient

$$\theta_{n+1} = \theta_n - \lambda \nabla f(\theta_n)$$

learning rate

in-class: implement gradient descent from scratch

$$f(x, y) = (x - 3)^2 + (y + 1)^2$$

1. write down the gradient analytically
2. implement update loop: $\theta_{n+1} = \theta_n - \lambda \nabla f(\theta_n)$
3. starting from (0,0), try $\lambda = 0.01, 0.1, 0.5, 1.1$ for N steps
4. plot the path taken across iterations on top of f

gradient descent: Adaptive Moment Estimation(Adam ; *Kingma & Ba 2014*)

$$\theta_{n+1} = \theta_n - \lambda \nabla f(\theta_n)$$



$$\theta_{n+1} = \theta_n - \lambda \frac{\langle \nabla f(\theta_n) \rangle}{\sqrt{\langle (\nabla f(\theta_n))^2 \rangle}}$$

gradient descent: Adaptive Moment Estimation(Adam ; *Kingma & Ba 2014*)

$$\theta_{n+1} = \theta_n - \lambda \nabla f(\theta_n)$$



$$\theta_{n+1} = \theta_n - \lambda \frac{\langle \nabla f(\theta_n) \rangle}{\sqrt{\langle (\nabla f(\theta_n))^2 \rangle}}$$

rolling average of gradients
(*momentum*)

gradient descent: Adaptive Moment Estimation(Adam ; *Kingma & Ba 2014*)

$$\theta_{n+1} = \theta_n - \lambda \nabla f(\theta_n)$$



$$\theta_{n+1} = \theta_n - \lambda \frac{\langle \nabla f(\theta_n) \rangle}{\sqrt{\langle (\nabla f(\theta_n))^2 \rangle}}$$

rolling average of gradients
(*momentum*)

volatility of the gradient

gradient descent: Adaptive Moment Estimation(Adam ; *Kingma & Ba 2014*)

$$\theta_{n+1} = \theta_n - \lambda \nabla f(\theta_n)$$



$$\theta_{n+1} = \theta_n - \lambda \frac{\langle \nabla f(\theta_n) \rangle}{\sqrt{\langle (\nabla f(\theta_n))^2 \rangle}}$$

rolling average of gradients
(*momentum*)

volatility of the gradient

Adam and its variants are the default optimizers for training neural networks

beyond first order: **Newton's method**

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \nabla f(\theta_0)^T (\theta - \theta_0)$$

gradient

only keep the first order

beyond first order: **Newton's method**

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \underbrace{\nabla f(\theta_0)^T}_{\text{gradient}} (\theta - \theta_0) + \frac{1}{2} (\theta - \theta_0)^T \underbrace{\mathbf{H}f(\theta_0)}_{\text{hessian}} (\theta - \theta_0) + \mathcal{O}((\theta - \theta_0)^3)$$

beyond first order: **Newton's method**

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \underbrace{\nabla f(\theta_0)^T}_{\text{gradient}} (\theta - \theta_0) + \frac{1}{2} (\theta - \theta_0)^T \underbrace{\mathbf{H} f(\theta_0)}_{\text{hessian}} (\theta - \theta_0) + \mathcal{O}((\theta - \theta_0)^3)$$

$$\theta_{n+1} = \theta_n - \mathbf{H}^{-1} \nabla f(\theta_n)$$


beyond first order: **Newton's method**

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \underbrace{\nabla f(\theta_0)^T}_{\text{gradient}} (\theta - \theta_0) + \frac{1}{2} (\theta - \theta_0)^T \underbrace{\mathbf{H} f(\theta_0)}_{\text{hessian}} (\theta - \theta_0) + \mathcal{O}((\theta - \theta_0)^3)$$

$$\theta_{n+1} = \theta_n - \mathbf{H}^{-1} \nabla f(\theta_n)$$

no learning rate!

beyond first order: **Newton's method**

taylor expand $f(\theta)$ around θ_0

$$f(\theta) = f(\theta_0) + \underbrace{\nabla f(\theta_0)^T}_{\text{gradient}} (\theta - \theta_0) + \frac{1}{2} (\theta - \theta_0)^T \underbrace{\mathbf{H} f(\theta_0)}_{\text{hessian}} (\theta - \theta_0) + \mathcal{O}((\theta - \theta_0)^3)$$

$$\theta_{n+1} = \theta_n - \mathbf{H}^{-1} \nabla f(\theta_n)$$

no learning rate!

$\mathbf{H}$ costs $\mathcal{O}(N^2)$ and $\mathcal{O}(N^3)$ to invert

quasi-Newton method: Broyden-Fletcher-Goldfarb-Shanno (**BFGS**)

approximates **H** using changes in the gradient from past steps

~~$$\theta_{n+1} = \theta_n - \mathbf{H}^{-1} \nabla f(\theta_n)$$~~

$$\theta_{n+1} = \theta_n - \mathbf{B}_n \nabla f(\theta_n)$$

quasi-Newton method: Broyden-Fletcher-Goldfarb-Shanno (**BFGS**)

approximates $\mathbf{H}$ using changes in the gradient from past steps

~~$$\theta_{n+1} = \theta_n - \mathbf{H}^{-1} \nabla f(\theta_n)$$~~

$$\theta_{n+1} = \theta_n - \mathbf{B}_n \nabla f(\theta_n)$$

$$s_n = \theta_{n+1} - \theta_n$$

$$y_n = \nabla f(\theta_{n+1}) - \nabla f(\theta_n)$$

$$\mathbf{B}_{n+1} = (I - \rho_n s_n y_n^T) \mathbf{B}_n (I - \rho_n y_n s_n^T) + \rho_n s_n s_n^T$$

$$\rho_n = \frac{1}{y_n^T s_n}$$

quasi-Newton method: Broyden-Fletcher-Goldfarb-Shanno (**BFGS**)

approximates $\mathbf{H}$ using changes in the gradient from past steps

~~$$\theta_{n+1} = \theta_n - \mathbf{H}^{-1} \nabla f(\theta_n)$$~~

$$\theta_{n+1} = \theta_n - \mathbf{B}_n \nabla f(\theta_n)$$

$$s_n = \theta_{n+1} - \theta_n$$

$$y_n = \nabla f(\theta_{n+1}) - \nabla f(\theta_n)$$

$$\mathbf{B}_{n+1} = (I - \rho_n s_n y_n^T) \mathbf{B}_n (I - \rho_n y_n s_n^T) + \rho_n s_n s_n^T$$

$$\rho_n = \frac{1}{y_n^T s_n}$$

satisfies the secant equation $\mathbf{B}_{n+1} y_n = s_n$ — *i.e.* $\mathbf{B}_{n+1}$ correctly maps the change in gradients to change in position

quasi-Newton method: Broyden-Fletcher-Goldfarb-Shanno (**BFGS**)
`scipy.optimize.minimize(method='BFGS')`

approximates **H** using changes in the gradient from past steps

$$\cancel{\theta_{n+1} = \theta_n - \mathbf{H}^{-1} \nabla f(\theta_n)}$$

$$\theta_{n+1} = \theta_n - \mathbf{B}_n \nabla f(\theta_n)$$

$$s_n = \theta_{n+1} - \theta_n$$

$$y_n = \nabla f(\theta_{n+1}) - \nabla f(\theta_n)$$

$$\mathbf{B}_{n+1} = (I - \rho_n s_n y_n^T) \mathbf{B}_n (I - \rho_n y_n s_n^T) + \rho_n s_n s_n^T$$

$$\rho_n = \frac{1}{y_n^T s_n}$$

satisfies the secant equation $\mathbf{B}_{n+1} y_n = s_n$ — *i.e.* $\mathbf{B}_{n+1}$ correctly maps the change in gradients to change in position

how do we get the gradients?

analytic (*e.g. in-class example*): exact but not feasible for complex $f(\theta)$

finite differences: approximate and requires extra evaluation per dimension

automatic differentiation (“autodiff”): JAX, PyTorch, etc all provide automatic differentiation

how do we get the gradients?

analytic (*e.g. in-class example*): exact but not feasible for complex $f(\theta)$

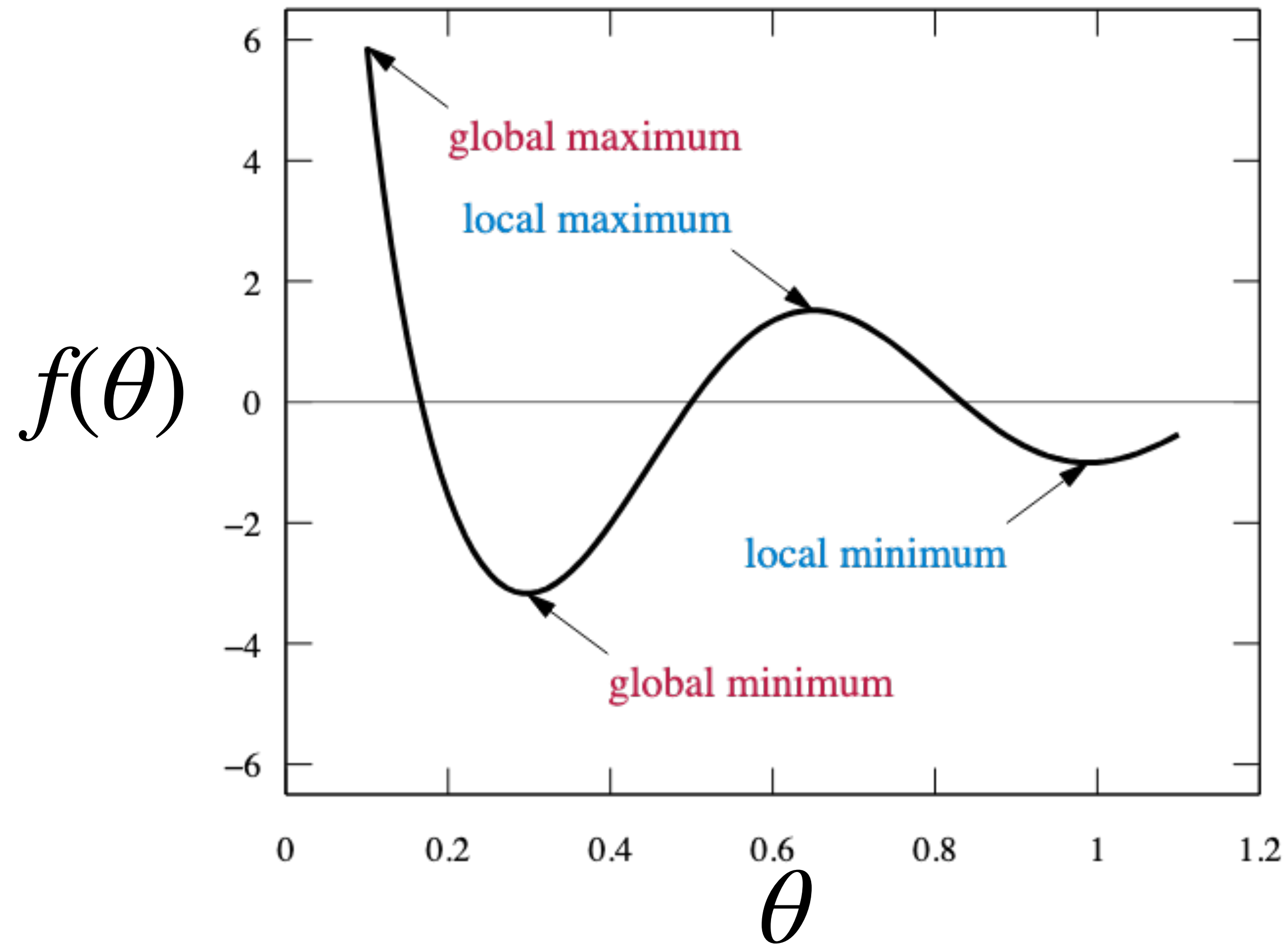
finite differences: approximate and requires extra evaluation per dimension

automatic differentiation (“autodiff”): JAX, PyTorch, etc all provide automatic differentiation

applying chain rule to elementary operations alongside function evaluations

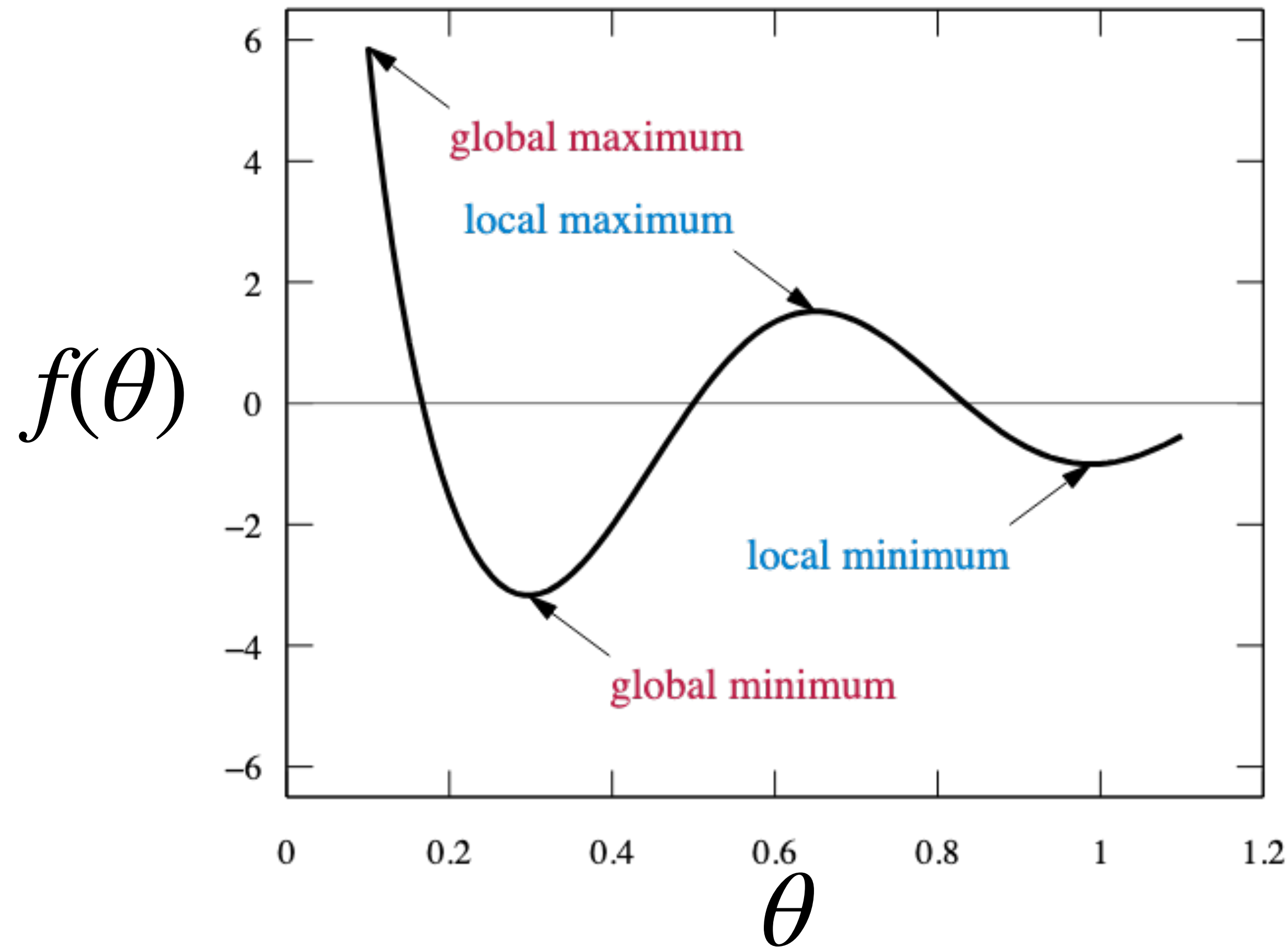
more on autodiff later!

local versus global minima



objective functions can have multiple local minima!

local versus global minima



optimizer reports: “the best point I could find, starting from where you put me”

local versus global minima

multi-start — multiple random initializations of the optimizer

a coarse search to locate a good starting region

random jumps during optimization (*unintentional advantage of stochastic gradient descent*)

in-class: escape the local minima!

$$f(x) = \sin(5x) + 0.1x^2$$

try running BFGS from 10 evenly spaced starting points across

$x \in [-10, 10]$ and plot the result

```
scipy.optimize.minimize(method='BFGS')
```

0th order: Nelder Mead

1st order: gradient descent (*e.g. Adam*)

2nd order: Newton, BFGS

local versus global minima

MLE and MAP are just a single point in the PDF — **next: Monte Carlo methods to sample the full posterior**